RoadMap of ProctoEase BackEnd

Multi-tenant AI-assisted online examination SaaS

This roadmap details the current status, remaining phases, execution plan, and technical priorities for the ProctoEase backend project, aligned with a multi-tenant SaaS architecture.1. Completion Status (Done)

The foundational and core identity/business logic phases are complete.

- ✅ **Phase 1 — Foundation (1 Day):** Docker infra, Pydantic config, async SQLAlchemy with RLS, JWT + bcrypt security, initial models (Tenant/User), Alembic, and health endpoint.
- ✅ **Phase 2 — Auth & Tenancy (1 Day):** Authentication schemas/services, get_current_user + require_role() RBAC, TenantMiddleware, and auth/tenant APIs (register, login, refresh, /me, create tenant).
- ✅ **Phase 3 — Business Endpoints (1 Day):** Exam + ExamAttempt models, RLS-enabled migration, exam/attempt schemas, services, and RBAC-protected routers. Enforces one active attempt per candidate.

2. Milestone Roadmap & Phase Breakdown (Remaining)

Phase	Focus Area	Effort (Days)	Complexity	Priority	Dependencies
4	**Hardening**	2	Medium	🔴 MVP-Critical	—
5	**AI Proctoring**	4	High	🟡 Core Feature	Phase 4
6	**Judge0 Code Execution**	3	Medium	🟡 Core Feature	Phase 4
7	**Plagiarism Detection**	3	High	🟢 Enhancement	Phase 6
8	**Risk Scoring Engine**	2	Medium	🟢 Enhancement	Phase 5
9	**Reporting & Analytics**	3	Medium	🟢 Enhancement	Phases 5–8
10	**Production Readiness**	2	Low	🔴 MVP-Critical	—
Total Remaining		**~22 days**			

Note on Dependencies: *Phases 5 (AI Proctoring) and 6 (Judge0 Code Execution) can run in parallel.*Phase Deliverables

Phase	Key Deliverables
4: Hardening	Custom exception classes, consistent global error handling, request validation edge case fixes, structured logging, smoke tests for all endpoints, Dockerfile optimization, API docs polish.
5: AI Proctoring	Proctoring event model and table, real-time WebSocket endpoint (ws://.../proctor), frame analysis/classification service, flagged frame storage, client SDK contract documentation.
6: Judge0 Code Execution	Judge0 Docker service integration, service for submitting code and polling results, models for code submission, API endpoint for submission, language whitelist config, timeout/memory limits.
7: Plagiarism Detection	Token-based similarity service, plagiarism report model, Celery background task for pairwise comparison, per-tenant similarity threshold configuration, and results API.
8: Risk Scoring Engine	Weighted score service (from proctoring events), configurable scoring model, per-attempt risk score model, real-time updates via Redis pub/sub, threshold alerts, and dashboard API.
9: Reporting & Analytics	Tenant dashboard API (exam stats, pass rates), per-exam analytics (avg score, completion), candidate performance history, CSV/PDF export endpoints.
10: Production Readiness	Redis-backed rate limiting, DB connection pooling tuning, health check expansion (Redis, Judge0), CI/CD pipeline, and secret rotation support, Prometheus metrics monitoring endpoint.

3. Week-by-Week Execution Plan

Week	Focus	Deliverables
Week 1	Foundation + Auth + Business	**Phases 1–3 ✅ (Done)**
Week 2	Hardening + AI Proctoring start	Error handling, tests, logging, WS scaffold
Week 3	AI Proctoring + Judge0 (parallel)	Frame analysis, code execution, submissions
Week 4	Plagiarism + Risk Scoring	Similarity engine, weighted scoring, alerts
Week 5	Reporting + Production Readiness	Analytics APIs, CI/CD, rate limiting

4. Kanban-Style Task List

Status	Tasks
✅ **Done**	Foundation, Auth & Tenancy, Business Endpoints
⏳ **Next**	Hardening: error handler, smoke tests, structlog, Dockerfile, custom exceptions.
📋 **Backlog**	AI Proctoring: WS endpoint, frame analysis. Judge0: Docker service, code submission API. Plagiarism: similarity engine. Risk Scoring: weighted model, Redis pub/sub. Production: CI/CD, monitoring.

🔮 **Future**	Reporting: CSV/PDF export. Real-time notifications, Admin superpanel, Question bank module.

5. Technical Priorities

The immediate technical priorities are defined by the **MVP-Critical** and **Core Feature** phases:

- 🔴 **MVP-Critical (Hardening & Production Readiness):** Focusing on stability, security, and operational readiness.
  - **Phase 4: Hardening:** Implementing robust custom exceptions, global error handling, comprehensive smoke tests, structured logging, and Docker optimization.
  - **Phase 10: Production Readiness:** Adding rate limiting, connection pooling tuning, full health checks, and a CI/CD pipeline.
- 🟡 **Core Feature (AI Proctoring & Judge0):** Delivering the main competitive features of the product.
  - **Phase 5: AI Proctoring:** Developing real-time event streaming and frame analysis.
  - **Phase 6: Judge0 Code Execution:** Integrating the code execution engine for technical assessments.

6. Potential Architectural Risks & Mitigation

Risk	Impact	Mitigation
WebSocket scaling	Proctoring WS connections × concurrent candidates	Horizontal scaling + connection limits per worker
Frame processing latency	ML inference blocking event loop	Offload to background worker (Celery/process pool)
Judge0 resource exhaustion	Students submitting infinite loops	Per-submission timeout + memory cap + queue limits
Refresh token theft	Stateless tokens can't be revoked	Phase 4: move to DB-backed token store + rotation
RLS bypass	Bug in get_db skips SET app.current_tenant_id	Unit test asserting RLS context is always set